

1. Requerimientos del proyecto

1.1 Requerimientos funcionales

RF1: Registro individual de animales

Debe permitir:

- RF1a: dar de alta a un nuevo animal.
- RF1b: consultar la ficha individual de cada animal.
- RF1c: editar la ficha de un animal.
- RF1d: desactivar un animal.

RF2: Gestión de alimentación

Debe permitir asignar planes alimenticios a cada rodeo.

RF3: Eventos sanitarios

Debe permitir:

- RF3a: registrar eventos sanitarios asociados a un animal.
- RF3b: registrar eventos sanitarios asociados a un rodeo.

RF4: Control de peso

Debe ser posible registrar el peso de cada animal de forma periódica.

RF5: Envío de alertas automáticas

El sistema debe enviar alertas mediante correo electrónico al usuario que corresponda ante las siguientes condiciones:

- RF5a: desvíos en la ganancia de peso esperada.
- RF5b: vencimientos de planes alimenticios.
- RF5c: vencimientos de eventos sanitarios.

RF6: Generación de reportes

El sistema debe permitir generar reportes diarios, semanales y mensuales, incluyendo la siguiente información:

- RF6a: inventario de animales.
- RF6b: fechas importantes a recordar.
- RF6c: evolución de peso.
- RF6d: consumo de alimentos (fuera del alcance de esta versión).

RF7: Usuarios y roles

El sistema debe soportar la existencia de diferentes tipos de usuarios (administrador, encargado de campo, asesor técnico), con permisos diferenciados para cada rol:

- RF7a: consultar lista de usuarios y roles.
- RF7b: crear usuarios (solo administrador).
- RF7c: editar usuarios (cambiar datos o rol).
- RF7d: eliminar usuario.

RF8: Gestión de catálogos

Debe permitir administrar los catálogos utilizados por los planes alimenticios y los eventos sanitarios:

- RF8a: alta, consulta, edición y desactivación de ingredientes.
- RF8b: alta, consulta, edición, desactivación y reactivación de medicamentos.

1.2 Requerimientos no funcionales

RNF1: Usabilidad

La interfaz debe ser sencilla, clara y fácil de usar, incluso con personas con poca experiencia en tecnología.

RNF2: Portabilidad

El sistema debe ser accesible desde computadora y dispositivos móviles (teléfono o tablet).

RNF3: Rendimiento

El sistema debe responder rápidamente, idealmente en menos de 2 a 5 segundos por acción.

RNF4: Seguridad

Los datos deben estar protegidos y cifrados, con acceso mediante usuario y contraseña.

RNF5: Mantenimiento

El sistema debe ser fácil de actualizar y mantener.

RNF6: Escalabilidad

El sistema debe permitir el incremento de usuarios y registros manteniendo tiempos de respuesta adecuados para el usuario.

RNF7: Confiabilidad

El sistema debe garantizar la integridad de los datos, evitando pérdidas de información y asegurando una disponibilidad mínima del 99% del tiempo.

2. Especificaciones de casos de uso

CU1 — Alta de nuevo animal

Actor principal: Encargado de campo / Administrador

Propósito: Registrar un nuevo animal en el sistema.

Precondiciones: Actor con permisos de alta.

Postcondiciones: Animal registrado; datos persistidos y visibles en la ficha.

Datos (entrada / pantalla / salida):

Entrada:

- D1: caravana_cuig: string (obligatorio)
- D2: caravana_nro_manejo: string (obligatorio; junto con D1 forma la clave compuesta de la caravana, única en el sistema)
- D3: fecha_nacimiento: date (obligatorio)
- D4: peso_al_nacer: decimal (obligatorio)
- D5: caravana_cuig_madre: string (opcional; junto con D6 se resuelve a id_madre: int)
- D6: caravana_nro_manejo_madre: string (opcional; junto con D5 se resuelve a id_madre: int)
- D7: caravana_cuig_padre: string (opcional; junto con D8 se resuelve a id_padre: int)
- D8: caravana_nro_manejo_padre: string (opcional; junto con D7 se resuelve a id_padre: int)
- D9: sexo: enum {Macho, Hembra} (obligatorio)
- D10: raza: string (catálogo; obligatorio)
- D11: color_pelaje: string (opcional)
- D12: fecha_alta: date (autocompletada por el sistema al guardar)

Salida mostrada (tras guardar):

- S1: id_animal: int asignado
- S2: Resumen de ficha: caravana (caravana_cuig + caravana_nro_manejo), sexo, raza, fecha_nacimiento, peso_al_nacer, madre/padre (si aplican), fecha_alta, estado.

Reglas de validación:

- R1: caravana_cuig y caravana_nro_manejo no vacíos; cada uno alfanumérico hasta 5 caracteres. La combinación de ambos debe ser única en el sistema.
- R2: fecha_nacimiento menor o igual a la fecha actual.
- R3: peso_al_nacer mayor a 0 y menor o igual a 100 kg.
- R4: La caravana de la madre (D5 + D6) y la del padre (D7 + D8) se informan completas o no se informan. Si se informan, deben existir animales con esas caravanas; no pueden ser iguales entre sí ni iguales a la caravana del nuevo animal.
- R5: fecha_alta la establece el sistema (no editable por el usuario).

Flujo principal:

1. El sistema muestra el formulario de alta con los campos D1–D11.
2. El actor completa los datos y presiona Guardar.
3. El sistema valida R1–R5 y resuelve madre/padre por clave compuesta hacia id_madre e id_padre.
4. El sistema persiste el nuevo animal, asigna id_animal y registra fecha_alta.
5. El sistema confirma "Alta exitosa" y muestra la ficha (S1–S2).

Excepciones / Alternativos:

- E1: Caravana duplicada (R1): informar "La caravana ya existe"; permitir corregir (volver a paso 1).
- E2: Campos inválidos (formato de R1, R2–R3): marcar el campo con mensaje específico; permitir corregir (volver a paso 1).
- E3: Madre o padre incompletos, inexistentes o inconsistentes (R4): informar detalle; permitir corregir (volver a paso 1).

CU2 — Consulta de ficha individual de un animal

Actor principal: Encargado de campo / Administrador

Propósito: Visualizar la ficha completa de un animal registrado en el sistema.

Precondiciones: Actor con permisos de consulta. Existe al menos un animal registrado.

Postcondiciones: Ficha del animal mostrada en pantalla.

Datos (entrada / pantalla / salida):**Entrada:**

- D1: caravana_cuig: string (obligatorio)
- D2: caravana_nro_manejo: string (obligatorio)

Salida mostrada (ficha):

- S1: caravana: string (caravana_cuig + caravana_nro_manejo)
- S2: fecha_nacimiento: date
- S3: peso_al_nacer: decimal
- S4: sexo: enum {Macho, Hembra}
- S5: raza: string
- S6: color_pelaje: string
- S7: fecha_alta: date
- S8: estado: string
- S9: rodeo_actual: string (si posee)
- S10: madre / padre: caravana (clave compuesta, si existen)
- S11: ultimo_peso: {fecha_pesaje: date, kg: decimal} (solo si existe al menos un registro de peso; indica qué tan reciente es el dato)

Reglas de validación:

- R1: caravana_cuig y caravana_nro_manejo no vacíos; cada uno alfanumérico hasta 5 caracteres.
- R2: Debe existir un animal con esa caravana (clave compuesta) en el sistema.

Flujo principal:

1. El sistema muestra los campos de búsqueda D1 y D2.
2. El actor ingresa la caravana y confirma la consulta.
3. El sistema valida R1 y R2.
4. El sistema recupera y muestra la ficha con los campos S1–S11.
5. Fin.

Excepciones / Alternativos:

- E1: Caravana inexistente (R2): informar "No se encontró un animal con esa caravana"; permitir reintento (volver a paso 1).

- E2: Formato inválido (R1): informar "Formato de caravana inválido (cada parte alfanumérica, hasta 5 caracteres)"; permitir corregir (volver a paso 1).

CU3 — Edición de ficha individual de un animal

Actor principal: Encargado de campo / Administrador

Propósito: Actualizar los datos de un animal existente en el sistema.

Precondiciones: Actor con permisos de edición. Existe al menos un animal registrado.

Postcondiciones: Datos actualizados y visibles en la ficha del animal.

Datos (entrada / pantalla / salida):

Entrada para búsqueda:

- D1: caravana_cuig: string (obligatorio)
- D2: caravana_nro_manejo: string (obligatorio)

Campos editables:

- D3: fecha_nacimiento: date (obligatorio)
- D4: peso_al_nacer: decimal (obligatorio)
- D5: caravana_cuig_madre: string (opcional; junto con D6 se resuelve a id_madre: int)
- D6: caravana_nro_manejo_madre: string (opcional; junto con D5 se resuelve a id_madre: int)
- D7: caravana_cuig_padre: string (opcional; junto con D8 se resuelve a id_padre: int)
- D8: caravana_nro_manejo_padre: string (opcional; junto con D7 se resuelve a id_padre: int)
- D9: sexo: enum {Macho, Hembra} (obligatorio)
- D10: raza: string (catálogo; obligatorio)
- D11: color_pelaje: string (opcional)
- D12: caravana y fecha_alta: no editables en este CU

Salida mostrada (tras guardar):

- S1: caravana: string (caravana_cuig + caravana_nro_manejo)
- S2: fecha_nacimiento: date
- S3: peso_al_nacer: decimal

- S4: sexo: enum {Macho, Hembra}
- S5: raza: string
- S6: color_pelaje: string
- S7: fecha_alta: date
- S8: estado: string
- S9: rodeo_actual: string (si posee)
- S10: madre / padre: caravana (clave compuesta, si existen)
- S11: ultimo_peso: {fecha_pesaje: date, kg: decimal} (solo si existe al menos un registro de peso)

Reglas de validación:

- R1: caravana_cuig y caravana_nro_manejo (búsqueda) no vacíos; cada uno alfanumérico hasta 5 caracteres.
- R2: Debe existir un animal con esa caravana (clave compuesta) en el sistema.
- R3: fecha_nacimiento menor o igual a la fecha actual.
- R4: peso_al_nacer mayor a 0 y menor o igual a 100 kg.
- R5: La caravana de la madre (D5 + D6) y la del padre (D7 + D8) se informan completas o no se informan. Si se informan, deben existir animales con esas caravanas; no pueden ser iguales a la caravana del animal editado ni entre sí.
- R6: caravana y fecha_alta no son editables en este CU.

Flujo principal:

1. El sistema solicita la caravana (D1 y D2).
2. El actor ingresa la caravana.
3. El sistema valida R1 y R2, y carga la ficha con los campos editables D3–D11.
4. El actor actualiza los campos y presiona Guardar.
5. El sistema valida R3–R6 y resuelve madre/padre por clave compuesta hacia id_madre e id_padre.
6. El sistema persiste los cambios y muestra la ficha actualizada S1–S11.

Excepciones / Alternativos:

- E1: Caravana inexistente (R2): informar "No se encontró un animal con esa caravana"; permitir reintentar (volver a paso 1).

- E2: Formato inválido (R1): informar "Formato de caravana inválido (cada parte alfanumérica, hasta 5 caracteres)"; permitir corregir (volver a paso 1).
- E3: Campos inválidos (R3–R5): marcar el campo con mensaje específico; permitir corregir (volver a paso 4).

CU4 — Desactivar ficha de un animal

Actor principal: Encargado de campo / Administrador

Propósito: Desactivar un animal previamente dado de alta para que quede inactivo en el sistema.

Precondiciones: Actor con permisos de administración. Existe al menos un animal con estado Activo.

Postcondiciones: El animal queda Inactivo y se mueve al listado de animales inactivos.

Datos (entrada / pantalla / salida):

Entrada para búsqueda:

- D1: caravana_cuig: string (obligatorio)
- D2: caravana_nro_manejo: string (obligatorio)

Confirmación:

- D3: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "El animal fue desactivado correctamente."
- S2: Ficha del animal con estado: Inactivo.

Reglas de validación:

- R1: caravana_cuig y caravana_nro_manejo no vacíos; cada uno alfanumérico hasta 5 caracteres.
- R2: Debe existir un animal con esa caravana (clave compuesta) en el sistema.
- R3: El animal debe estar en estado Activo para poder desactivarse.
- R4: La desactivación marca al animal como Inactivo; su historial se conserva íntegro.

Flujo principal:

1. El sistema solicita la caravana (D1 y D2).
2. El actor ingresa la caravana.

3. El sistema valida R1, R2 y R3, y muestra un resumen de la ficha con el botón Desactivar.
4. El actor presiona Desactivar; el sistema solicita confirmación (D3).
5. El actor confirma.
6. El sistema aplica R4, marca al animal como Inactivo y muestra S1 y S2.

Excepciones / Alternativos:

- E1: Caravana inexistente (R2): informar "No se encontró un animal con esa caravana"; permitir reintentar (volver a paso 1).
- E2: Formato inválido (R1): informar "Formato de caravana inválido (cada parte alfanumérica, hasta 5 caracteres)"; permitir corregir (volver a paso 1).
- E3: Animal ya inactivo (R3): informar "El animal ya se encuentra inactivo en el sistema"; no se realizan cambios.
- E4: Operación cancelada (D3 = false): no se realizan cambios; volver a la vista anterior.

CU5 — Reactivación de ficha de un animal

Actor principal: Administrador

Propósito: Reactivar un animal previamente dado de baja para que vuelva a estar disponible en el sistema.

Precondiciones: Actor con permisos de administración. Existe al menos un animal con estado Inactivo.

Postcondiciones: El animal queda activo y vuelve a aparecer en listados y operaciones.

Datos (entrada / pantalla / salida):

Entrada para búsqueda:

- D1: caravana_cuig: string (obligatorio)
- D2: caravana_nro_manejo: string (obligatorio)

Confirmación:

- D3: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "El animal fue reactivado correctamente."
- S2: Ficha del animal con estado: Activo.

Reglas de validación:

- R1: caravana_cuig y caravana_nro_manejo no vacíos; cada uno alfanumérico hasta 5 caracteres.
- R2: Debe existir un animal con esa caravana (clave compuesta) en el sistema.
- R3: El animal debe estar en estado Inactivo para poder reactivarse.
- R4: La reactivación restaura el estado del animal a Activo; su historial se conserva íntegro.
- R5: Solo el Administrador puede realizar esta operación.

Flujo principal:

1. El sistema solicita la caravana (D1 y D2).
2. El actor ingresa la caravana.
3. El sistema valida R1, R2 y R3, y muestra un resumen de la ficha con el botón Reactivar.
4. El actor presiona Reactivar; el sistema solicita confirmación (D3).
5. El actor confirma.
6. El sistema aplica R4, marca al animal como Activo y muestra S1 y S2.

Excepciones / Alternativos:

- E1: Caravana inexistente (R2): informar "No se encontró un animal con esa caravana"; permitir reintentar (volver a paso 1).
- E2: Formato inválido (R1): informar "Formato de caravana inválido (cada parte alfanumérica, hasta 5 caracteres)"; permitir corregir (volver a paso 1).
- E3: Animal ya activo (R3): informar "El animal ya se encuentra activo en el sistema"; no se realizan cambios.
- E4: Operación cancelada (D3 = false): no se realizan cambios; volver a la vista anterior.

CU6 — Crear un nuevo rodeo

Actor principal: Encargado de campo / Administrador

Propósito: Crear un rodeo y asignarle animales.

Precondiciones: Actor con permisos de creación. Existen al menos dos animales activos sin rodeo asignado.

Postcondiciones: Rodeo creado y visible en la lista; animales seleccionados asignados al rodeo.

Datos (entrada / pantalla / salida):

Entrada:

- D1: nombre_rodeo: string (obligatorio)
- D2: descripcion: string (opcional)
- D3: Selector de animales activos sin rodeo asignado; búsqueda por caravana; selección múltiple.

Salida tras crear:

- S1: Mensaje: "Rodeo creado con éxito."
- S2: nombre_rodeo: string
- S3: cantidad_animales_asignados: int

Reglas de validación:

- R1: nombre_rodeo no vacío y no repetido entre los rodeos activos (sin distinguir mayúsculas y minúsculas).
- R2: Deben seleccionarse dos o más animales.
- R3: Solo pueden seleccionarse animales activos sin rodeo asignado.
- R4: Tipos de datos válidos para nombre_rodeo y descripcion.

Flujo principal:

1. El sistema muestra el formulario con D1, D2 y D3.
2. El actor completa el nombre del rodeo, opcionalmente la descripción, selecciona dos o más animales y presiona Crear.
3. El sistema valida R1–R4.
4. El sistema crea el rodeo y asigna los animales seleccionados.
5. El sistema muestra S1, S2 y S3.

Excepciones / Alternativos:

- E1: Nombre de rodeo duplicado (R1): informar "Ya existe un rodeo con ese nombre"; permitir corregir (volver a paso 2).
- E2: Selección insuficiente (R2): informar "Debe seleccionar al menos dos animales"; permitir corregir (volver a paso 2).

- E3: Animal no disponible (R3): informar "Uno o más animales seleccionados no están disponibles"; no agregarlos y permitir continuar con el resto.

CU7 — Transferir animales entre rodeos

Actor principal: Encargado de campo / Administrador

Propósito: Mover uno o más animales de su rodeo actual a otro rodeo en una única operación.

Precondiciones: Actor con permisos de edición. Existen al menos dos rodeos creados y al menos un animal activo asignado a un rodeo.

Postcondiciones: Los animales seleccionados quedan asignados al rodeo destino y desvinculados del rodeo origen.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_rodeo_origen: int (resuelto por la pantalla)
- D2: Lista de animales del rodeo origen; identificados por caravana; selección múltiple.
- D3: id_rodeo_destino: int (obligatorio; seleccionado de la lista de rodeos disponibles)

Salida tras confirmar:

- S1: Mensaje: "Transferencia realizada con éxito."
- S2: cantidad_animales_transferidos: int
- S3: nombre_rodeo_destino: string

Reglas de validación:

- R1: id_rodeo_origen debe existir y tener al menos un animal activo.
- R2: Debe seleccionarse al menos un animal.
- R3: id_rodeo_destino debe existir y ser distinto al id_rodeo_origen.
- R4: Los animales seleccionados deben pertenecer al rodeo origen al momento de la operación.
- R5: La operación es atómica; si falla alguna transferencia individual no se aplica ningún cambio.

Flujo principal:

1. El sistema muestra la lista de rodeos disponibles para seleccionar el rodeo origen.
2. El actor selecciona el rodeo origen; el sistema muestra los animales que lo componen con D2.
3. El actor selecciona uno o más animales y elige el rodeo destino D3.
4. El actor presiona Transferir; el sistema solicita confirmación.
5. El actor confirma.
6. El sistema valida R1–R5 y actualiza el rodeo de cada animal seleccionado.
7. El sistema muestra S1, S2 y S3.

Excepciones / Alternativos:

- E1: Rodeo origen o destino inexistente (R1, R3): informar el problema; permitir corregir.
- E2: Rodeo destino igual al origen (R3): informar "El rodeo destino debe ser diferente al rodeo origen"; permitir corregir.
- E3: Ningún animal seleccionado (R2): informar "Debe seleccionar al menos un animal"; permitir corregir.
- E4: Operación cancelada: no se realizan cambios; volver a la vista anterior.

CU8 — Eliminar un rodeo

Actor principal: Encargado de campo / Administrador

Propósito: Quitar un rodeo del sistema.

Precondiciones: Actor con permisos de eliminación. Existe al menos un rodeo creado.

Postcondiciones: Rodeo marcado como inactivo (activo = false) y removido de los listados, sin eliminarse físicamente; los animales quedan sin rodeo asignado; las asignaciones de planes alimenticios del rodeo quedan marcadas como inactivas, conservando el historial.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_rodeo: int (resuelto por la pantalla)

Confirmación:

- D2: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "Rodeo eliminado con éxito."

Reglas de validación:

- R1: id_rodeo debe existir en el sistema.
- R2: Antes de solicitar confirmación el sistema muestra un resumen con la cantidad de animales y la cantidad de planes alimenticios asignados al rodeo.
- R3: La eliminación es lógica: al confirmar, el rodeo se marca como inactivo (activo = false) y su registro no se borra; los animales quedan con id_rodeo nulo (no se eliminan), y las asignaciones de PlanRodeoAsignacion asociadas al rodeo se marcan como inactivas (activo = false), conservando el historial.

Flujo principal:

1. El sistema muestra el rodeo seleccionado con el botón Eliminar.
2. El actor presiona Eliminar; el sistema muestra el resumen indicado en R2 y solicita confirmación (D2).
3. El actor confirma.
4. El sistema valida R1, aplica R3 y elimina el rodeo.
5. El sistema muestra S1.

Excepciones / Alternativos:

- E1: Operación cancelada (D2 = false): no se realizan cambios; volver a la lista de rodeos.
- E2: Rodeo inexistente (R1): informar "No se encontró el rodeo seleccionado"; volver a la lista de rodeos.

CU9 — Crear plan alimenticio

Actor principal: Asesor técnico / Administrador

Propósito: Definir un plan alimenticio con su fórmula y componentes, listo para ser asignado a los rodeos.

Precondiciones: Sesión iniciada. Actor con permisos de creación de planes.

Postcondiciones: Plan alimenticio creado y disponible para ser asignado a uno o más rodeos.

Datos (entrada / pantalla / salida):**Entrada:**

- D1: nombre_plan: string (obligatorio)
- D2: categoria: string (opcional)
- D3: peso_vivo_inicial_promedio: decimal (kg; opcional)
- D4: peso_objetivo: decimal (kg; opcional)
- D5: ganancia_peso_esperada: decimal (kg/día; opcional)
- D6: tipo_alimentacion: string (obligatorio)
- D7: tiempo_alimentacion: string (opcional)
- D8: kg_ms_diaria_por_animal: decimal (obligatorio)
- D9: observaciones: string (opcional)
- D10: DetallePlan (obligatorio; uno o más componentes)
 - D10.1: id_ingredientes: int (obligatorio; seleccionado del catálogo de ingredientes activos)
 - D10.2: porcentaje_inclusion_ms: decimal (obligatorio)
 - D10.3: observaciones: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Plan alimenticio creado con éxito."

Reglas de validación:

- R1: nombre_plan no vacío y no repetido en el sistema (sin distinguir mayúsculas y minúsculas).
- R2: tipo_alimentacion no vacío.
- R3: kg_ms_diaria_por_animal mayor a 0.
- R4: Debe informarse al menos un componente. Por cada componente: el ingrediente debe existir en el catálogo y estar activo, y porcentaje_inclusion_ms debe estar entre 0 y 100.
- R5: Un mismo ingrediente no puede repetirse dentro del mismo plan.
- R6: La suma de porcentaje_inclusion_ms de todos los componentes no puede superar 100.

Flujo principal:

1. El sistema muestra el formulario con D1–D9 y la grilla de componentes D10.
2. El actor completa los datos del plan, agrega los componentes que correspondan y presiona Guardar.
3. El sistema valida R1–R6.
4. El sistema crea el plan y lo guarda en la base de datos.
5. El sistema muestra S1.

Excepciones / Alternativos:

- E1: Nombre duplicado (R1): informar "Ya existe un plan con ese nombre"; permitir corregir.
- E2: Datos inválidos (R2–R4): marcar el campo a corregir con mensaje específico; permitir reintentar.
- E3: Ingrediente duplicado (R5): informar "El ingrediente ya fue agregado al plan"; no agregarlo y permitir corregir.
- E4: Suma de porcentaje_inclusion_ms supera 100 (R6): informar "La suma de los porcentajes de inclusión supera el 100%; ajuste los valores antes de continuar."

CU10 — Asignar plan alimenticio a un rodeo

Actor principal: Asesor técnico / Administrador

Propósito: Asignar un plan alimenticio existente a un rodeo, definiendo los datos operativos de la asignación.

Precondiciones: Sesión iniciada. Existe al menos un plan alimenticio creado y al menos un rodeo con animales asignados.

Postcondiciones: Plan alimenticio asignado al rodeo con los datos operativos correspondientes.

Datos (entrada / pantalla / salida):**Entrada:**

- D1: id_plan_alimenticio: int (obligatorio; seleccionado de la lista de planes disponibles)
- D2: id_rodeo: int (obligatorio; seleccionado de la lista de rodeos disponibles)
- D3: vigencia_desde: date (obligatorio)

- D4: vigencia_hasta: date (opcional)

Salida tras guardar:

- S1: Mensaje: "Plan alimenticio asignado con éxito."
- S2: nombre_plan: string
- S3: nombre_rodeo: string
- S4: vigencia_desde: date
- S5: vigencia_hasta: date
- S6: cant_animales: int (calculado por el sistema: cantidad de animales activos del rodeo al momento de la asignación; no editable)
- S7: kg_ms_diaria_total: decimal (calculado por el sistema: $\text{kg_ms_diaria_por_animal} \times \text{cant_animales}$)
- S8: nombre_plan_anterior_reemplazado: string (solo si el rodeo tenía otro plan vigente)

Reglas de validación:

- R1: id_plan_alimenticio debe existir en el sistema.
- R2: id_rodeo debe existir en el sistema y tener al menos un animal activo asignado.
- R3: vigencia_desde no vacía y con formato válido.
- R4: Si se informa vigencia_hasta, debe ser igual o posterior a vigencia_desde.
- R5: cant_animales y kg_ms_diaria_total se calculan automáticamente; no son editables por el actor.
- R6: Si el rodeo ya tiene asignado ese mismo plan de forma activa, no se vuelve a asignar.
- R7: Si el rodeo tiene otro plan asignado de forma activa, esa asignación se cierra (activo = false y vigencia_hasta = vigencia_desde de la nueva asignación), conservando el historial, y su nombre se informa en S8.

Flujo principal:

1. El sistema muestra el formulario con D1–D4.
2. El actor selecciona el plan y el rodeo.
3. El actor completa los datos restantes y presiona Guardar.
4. El sistema valida R1–R6 y, si corresponde, aplica R7.

5. El sistema calcula cant_animales y kg_ms_diaria_total (R5), registra la asignación y muestra S1–S8.

Excepciones / Alternativos:

- E1: Plan inexistente (R1): informar "No se encontró el plan seleccionado"; permitir corregir.
- E2: Rodeo inexistente o sin animales (R2): informar "El rodeo seleccionado no existe o no tiene animales asignados"; permitir corregir.
- E3: Datos inválidos (R3, R4): marcar el campo a corregir con mensaje específico; permitir reintentar.
- E4: Plan ya asignado al rodeo (R6): informar "El plan ya está asignado a este rodeo actualmente"; no se realizan cambios.

CU11 — Editar plan alimenticio

Actor principal: Asesor técnico / Administrador

Propósito: Actualizar la fórmula y los componentes de un plan alimenticio existente.

Precondiciones: Sesión iniciada. Existe al menos un plan alimenticio creado.

Postcondiciones: Plan alimenticio actualizado y los cambios se reflejan en todas las asignaciones asociadas.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_plan_alimenticio: int (obligatorio)
- D2: nombre_plan: string (obligatorio)
- D3: categoria: string (opcional)
- D4: peso_vivo_inicial_promedio: decimal (kg; opcional)
- D5: peso_objetivo: decimal (kg; opcional)
- D6: ganancia_peso_esperada: decimal (kg/día; opcional)
- D7: tipo_alimentacion: string (obligatorio)
- D8: tiempo_alimentacion: string (opcional)
- D9: kg_ms_diaria_por_animal: decimal (obligatorio)
- D10: observaciones: string (opcional)
- D11: DetallePlan (obligatorio; uno o más componentes)

- D11.1: id_ingrediente: int (obligatorio; seleccionado del catálogo de ingredientes activos)
- D11.2: porcentaje_inclusion_ms: decimal (obligatorio)
- D11.3: observaciones: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Plan alimenticio actualizado con éxito."

Reglas de validación:

- R1: id_plan_alimenticio debe existir en el sistema.
- R2: nombre_plan no vacío y no repetido excluyendo el propio plan (sin distinguir mayúsculas y minúsculas).
- R3: tipo_alimentacion no vacío.
- R4: kg_ms_diaria_por_animal mayor a 0.
- R5: Debe informarse al menos un componente. Por cada componente: el ingrediente debe existir en el catálogo y estar activo, y porcentaje_inclusion_ms debe estar entre 0 y 100.
- R6: Un mismo ingrediente no puede repetirse dentro del mismo plan.
- R7: La suma de porcentaje_inclusion_ms de todos los componentes no puede superar 100.
- R8: Si el plan tiene asignaciones vigentes, el sistema advierte al actor antes de guardar que los cambios impactarán en esas asignaciones.

Flujo principal:

1. El sistema muestra el formulario precompletado con D2–D11.
2. El actor ajusta los campos y/o agrega o quita componentes.
3. Si el plan tiene asignaciones vigentes, el sistema muestra una advertencia indicando cuántas asignaciones se verán afectadas (R8).
4. El actor confirma y presiona Guardar.
5. El sistema valida R1–R7.
6. El sistema persiste los cambios y muestra S1.

Excepciones / Alternativos:

- E1: Nombre duplicado (R2): informar "Ya existe un plan con ese nombre"; permitir corregir.
- E2: Datos inválidos (R3–R5): marcar el campo a corregir con mensaje específico; permitir reintentar.
- E3: Ingrediente duplicado (R6): informar "El ingrediente ya fue agregado al plan"; no agregarlo y permitir corregir.
- E4: Suma de porcentaje_inclusion_ms supera 100 (R7): informar "La suma de los porcentajes de inclusión supera el 100%; ajuste los valores antes de continuar."

CU12 — Registrar evento sanitario a múltiples animales

Actor principal: Encargado de campo / Asesor técnico / Administrador

Propósito: Registrar un mismo evento sanitario para uno o varios animales en una única operación.

Precondiciones: Sesión iniciada. Existen animales activos registrados. Actor con permisos para registrar eventos sanitarios.

Postcondiciones: Evento sanitario registrado individualmente para cada animal seleccionado y visible en su historial sanitario.

Datos (entrada / pantalla / salida):

Entrada:

- D1: modo_seleccion: string (obligatorio; opciones: Rodeo completo | Selección manual | Selección libre)
- D2: id_rodeo: int (obligatorio si modo_seleccion = Rodeo completo o Selección manual; no aplica en Selección libre)
- D3: lista_animales: lista de caravanas (clave compuesta) seleccionadas en pantalla (obligatoria si modo_seleccion = Selección manual o Selección libre)
- D4: tipo_evento: enum (obligatorio; conjunto cerrado: Vacunación, Desparasitación, Tratamiento, Refuerzo, Control, Otro). Los casos no contemplados en la lista se registran como "Otro" y se detallan en observaciones (D9).
- D5: fecha_evento: date (obligatorio)
- D6: DetalleMedicamento (opcional; cero, uno o más)

- D6.1: id_medicamento: int (obligatorio dentro del detalle; seleccionado del catálogo de medicamentos activos)
- D6.2: dosis: decimal (opcional)
- D6.3: unidad: enum {ml, l, mg, g, UI, cm3} (obligatoria si se informa dosis)
- D6.4: observaciones: string (opcional)
- D7: vigencia_hasta: date (opcional)
- D8: fecha_proxima_aplicacion: date (opcional)
- D9: observaciones: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Evento sanitario registrado con éxito."
- S2: cantidad_animales_alcanzados: int
- S3: tipo_evento: string
- S4: fecha_evento: date

Reglas de validación:

- R1: Si modo_seleccion = Rodeo completo o Selección manual, id_rodeo debe existir en el sistema y tener al menos un animal activo.
- R2: Si modo_seleccion = Selección manual o Selección libre, la lista de animales no puede estar vacía.
- R3: En Selección manual, todas las caravanas seleccionadas deben existir, estar activas y pertenecer al rodeo indicado en D2. En Selección libre, deben existir y estar activas, sin importar el rodeo al que pertenezcan.
- R4: Si modo_seleccion = Rodeo completo, el sistema toma automáticamente todos los animales activos del rodeo seleccionado.
- R5: No pueden existir caravanas duplicadas en la lista.
- R6: tipo_evento no vacío y dentro del conjunto cerrado de opciones permitidas (validado en código).
- R7: fecha_evento válida y no posterior a la fecha actual.
- R8: Por cada detalle informado, id_medicamento debe existir en el catálogo de medicamentos y estar activo.
- R9: Si se informa dosis, debe ser mayor a 0.

- R10: Si se informa dosis, debe informarse también unidad, y esta debe ser una de las opciones permitidas.
- R11: Si se informa vigencia_hasta, debe ser igual o posterior a fecha_evento.
- R12: Si se informa fecha_proxima_aplicacion, debe ser igual o posterior a fecha_evento.

Flujo principal:

1. El sistema muestra las opciones de selección "Rodeo completo", "Selección manual" y "Selección libre" (D1).
2. El actor elige el modo de selección.
3. Si el actor elige "Rodeo completo", el sistema muestra el selector de rodeos; el actor elige uno y el sistema toma automáticamente todos los animales activos de ese rodeo.
4. Si el actor elige "Selección manual", el sistema muestra el selector de rodeos; el actor elige uno y el sistema muestra el listado de animales activos de ese rodeo para que el actor elija uno o varios.
5. Si el actor elige "Selección libre", el sistema muestra el listado de todos los animales activos, con búsqueda por caravana, para que el actor elija uno o varios.
6. El actor completa los datos del evento, opcionalmente agrega uno o más medicamentos del catálogo al detalle, y presiona Guardar.
7. El sistema valida R1–R12 y asocia automáticamente el usuario con sesión iniciada a cada evento registrado.
8. El sistema registra un evento sanitario individual para cada animal seleccionado.
9. El sistema muestra S1–S4.

Excepciones / Alternativos:

- E1: Datos inválidos (R6, R7, R9–R12): marcar el campo a corregir con mensaje específico; permitir reintentar.
- E2: Rodeo inexistente o sin animales activos (R1): informar "No se encontró el rodeo seleccionado o no tiene animales activos"; permitir corregir.
- E3: Selección vacía, o animal inexistente, inactivo, no perteneciente al rodeo o repetido en la lista (R2, R3, R5): informar cuáles presentan error y permitir ajustar la selección.

- E4: Medicamento inexistente o desactivado (R8): informar cuál detalle presenta el error; permitir corregir.

CU13 — Registrar peso de un animal

Actor principal: Encargado de campo / Administrador

Propósito: Registrar el peso de un animal con su fecha correspondiente.

Precondiciones: Sesión iniciada. Existe al menos un animal activo registrado.

Postcondiciones: Registro de peso guardado y visible en el historial del animal.

Datos (entrada / pantalla / salida):

Entrada:

- D1: caravana_cuig: string (obligatorio)
- D2: caravana_nro_manejo: string (obligatorio)
- D3: peso_kg: decimal (obligatorio; tomado de la báscula y mostrado en pantalla)
- D4: fecha_pesaje: date (el sistema propone la fecha actual como valor por defecto; editable por el actor)
- D5: observaciones: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Peso registrado correctamente."
- S2: caravana: string (caravana_cuig + caravana_nro_manejo)
- S3: fecha_pesaje: date
- S4: peso_kg: decimal

Reglas de validación:

- R1: caravana_cuig y caravana_nro_manejo no vacíos; cada uno alfanumérico hasta 5 caracteres. Debe existir un animal activo con esa caravana (clave compuesta).
- R2: peso_kg mayor a 0.
- R3: fecha_pesaje no puede ser posterior a la fecha actual.

Flujo principal:

1. El sistema muestra el formulario con D1–D5 y propone la fecha actual en D4.
2. El actor ingresa la caravana manualmente (D1 y D2).

3. El sistema valida R1 y muestra los datos básicos del animal para confirmar que es el correcto.
4. El actor ingresa o confirma el peso tomado de la báscula, ajusta la fecha si es necesario y presiona Guardar.
5. El sistema valida R2 y R3 y persiste el registro.
6. El sistema muestra S1–S4.

Excepciones / Alternativos:

- E1: Caravana inexistente, inactiva o con formato inválido (R1): informar "No se encontró un animal activo con esa caravana"; permitir reintentar.
- E2: Error o ausencia de lectura de báscula: informar "No se pudo obtener el peso; ingrese el valor manualmente"; permitir reintentar.
- E3: peso_kg inválido (R2): informar "El peso debe ser mayor a 0"; permitir corregir.
- E4: fecha_pesaje posterior a la fecha actual (R3): informar "La fecha del pesaje no puede ser posterior a la fecha actual"; permitir corregir.

CU14 — Alerta por desvío en la ganancia de peso esperada

Actor principal: Temporizador

Propósito: Notificar automáticamente cuando un animal o rodeo se desvía de la ganancia de peso esperada según el plan alimenticio asignado.

Precondiciones: Existen registros de peso en el sistema. Existe al menos un plan alimenticio asignado a un rodeo con ganancia_peso_esperada informada. Hay destinatarios configurados para recibir alertas.

Postcondiciones: Correo de alerta enviado a los destinatarios configurados y envío registrado en el sistema.

Datos (entrada / pantalla / salida):

Datos utilizados por el sistema:

- D1: Registros de ControlPeso: id_animal, fecha_pesaje, peso_kg.
- D2: ganancia_peso_esperada del plan alimenticio asignado al rodeo.

Salida (correo):

- S1: Asunto: "Alerta: desvío en la ganancia de peso esperada."

- S2: Cuerpo: identificación del animal o rodeo, período evaluado, ganancia real calculada, ganancia esperada y descripción del desvío detectado.

Reglas de validación:

- R1: Para evaluar un animal se requieren al menos dos registros de peso dentro del período evaluado.
- R2: La ganancia real por animal se calcula como la diferencia entre el último y el anterior pesaje ajustada al tiempo transcurrido entre ambos.
- R3: Para un rodeo la ganancia real se calcula como el promedio de los animales que cumplan R1.
- R4: Se genera alerta cuando la ganancia real es menor a la ganancia esperada definida en el plan.
- R5: Si el plan alimenticio asignado no tiene ganancia_peso_esperada informada no se emite alerta para ese caso.
- R6: No se emiten alertas duplicadas para un mismo animal o rodeo en el mismo período evaluado.

Flujo principal:

1. El temporizador inicia la revisión automática.
2. El sistema reúne los registros de ControlPeso y la ganancia esperada de cada plan asignado.
3. El sistema calcula la ganancia real por animal según R2 y por rodeo según R3.
4. El sistema compara la ganancia real con la esperada según R4.
5. Para cada desvío detectado el sistema arma el correo con S1 y S2 y lo envía a los destinatarios configurados.
6. El sistema registra el envío de cada alerta.
7. Fin.

Excepciones / Alternativos:

- E1: Datos insuficientes (R1): no se emite alerta para ese animal; se registra el motivo.
- E2: Sin ganancia esperada informada (R5): no se emite alerta para ese caso; se registra el motivo.
- E3: Alerta duplicada (R6): no se emite; se registra que ya fue notificado en el período.

- E4: Sin destinatarios configurados: no se envía la alerta; se registra que no se pudo notificar.

CU15 — Generar alertas sanitarias automáticas

Actor principal: Temporizador

Propósito: Notificar automáticamente eventos sanitarios próximos o vencimientos de eventos sanitarios.

Precondiciones: Existen eventos sanitarios registrados en el sistema. Hay destinatarios configurados para recibir alertas.

Postcondiciones: Alertas enviadas a los destinatarios configurados y envíos registrados en el sistema.

Datos (entrada / pantalla / salida):

Datos utilizados por el sistema:

- D1: id_evento_sanitario: int
- D2: id_animal: int
- D3: tipo_evento: string
- D4: medicamentos del detalle: string (si los hay)
- D5: fecha_evento: date
- D6: fecha_proxima_aplicacion: date (opcional)
- D7: vigencia_hasta: date (opcional)
- D8: observaciones: string (opcional)

Salida (correo):

- S1: Asunto: "Alerta sanitaria."
- S2: Cuerpo: identificación del destino (caravana del animal), tipo_evento, medicamentos del detalle (si los hay), tipo de alerta (próxima aplicación o vencimiento), fecha correspondiente y observaciones si existen.

Reglas de validación:

- R1: Se genera alerta si fecha_proxima_aplicacion está dentro del rango de aviso configurado o si vigencia_hasta está dentro del rango de aviso configurado.
- R2: El rango de aviso es configurable por el sistema; el valor por defecto es 7 días.
- R3: Deben existir destinatarios configurados para recibir notificaciones.

- R4: No se envían alertas duplicadas para un mismo evento sanitario en el mismo período.

Flujo principal:

1. El temporizador inicia la ejecución del proceso.
2. El sistema obtiene todos los eventos sanitarios registrados.
3. El sistema filtra los eventos cuya fecha_proxima_aplicacion o vigencia_hasta esté dentro del rango de aviso definido en R2.
4. Para cada evento válido el sistema determina el tipo de alerta y arma el mensaje correspondiente con S1 y S2.
5. El sistema envía los correos a los destinatarios configurados.
6. El sistema registra el envío de cada alerta.
7. Fin.

Excepciones / Alternativos:

- E1: Sin eventos próximos o vencimientos en el rango: no se envía ninguna alerta; se registra el resultado.
- E2: Alerta duplicada (R4): no se envía; se registra que ya fue notificado.
- E3: Sin destinatarios configurados (R3): no se envía la alerta; se registra que no se pudo notificar.

CU16 — Generar alertas por vencimiento de planes alimenticios

Actor principal: Temporizador

Propósito: Notificar automáticamente cuando la asignación de un plan alimenticio a un rodeo está próxima a vencer.

Precondiciones: Existen asignaciones de planes alimenticios activas con vigencia_hasta informada. Hay destinatarios configurados para recibir alertas.

Postcondiciones: Alertas enviadas a los destinatarios configurados y envíos registrados en el sistema.

Datos (entrada / pantalla / salida):

Datos utilizados por el sistema:

- D1: id_asignacion: int
- D2: nombre_plan: string

- D3: nombre_rodeo: string
- D4: vigencia_desde: date
- D5: vigencia_hasta: date
- D6: cant_animales: int (animales activos del rodeo al momento de la revisión)

Salida (correo):

- S1: Asunto: "Alerta: vencimiento de plan alimenticio."
- S2: Cuerpo: nombre del plan, nombre del rodeo, vigencia_desde, vigencia_hasta, días restantes hasta el vencimiento y cantidad de animales del rodeo.

Reglas de validación:

- R1: Solo se evalúan asignaciones activas (activo = true) con vigencia_hasta informada.
- R2: Se genera alerta si vigencia_hasta está dentro del rango de aviso, es decir, entre la fecha actual y la fecha actual más el rango configurado.
- R3: El rango de aviso es el mismo que el de las alertas sanitarias (CU15); el valor por defecto es 7 días.
- R4: Deben existir destinatarios configurados para recibir notificaciones.
- R5: No se envían alertas duplicadas para una misma asignación en el mismo período.

Flujo principal:

1. El temporizador inicia la ejecución del proceso.
2. El sistema obtiene las asignaciones de planes alimenticios que cumplen R1.
3. El sistema filtra las asignaciones cuya vigencia_hasta esté dentro del rango de aviso según R2 y R3.
4. Para cada asignación válida el sistema arma el mensaje con S1 y S2.
5. El sistema envía los correos a los destinatarios configurados.
6. El sistema registra el envío de cada alerta.
7. Fin.

Excepciones / Alternativos:

- E1: Sin asignaciones próximas a vencer en el rango: no se envía ninguna alerta; se registra el resultado.
- E2: Alerta duplicada (R5): no se envía; se registra que ya fue notificado.

- E3: Sin destinatarios configurados (R4): no se envía la alerta; se registra que no se pudo notificar.

CU17 — Generar reporte de inventario de animales

Actor principal: Administrador

Propósito: Obtener el inventario de animales por rodeo o en general para un período determinado.

Precondiciones: Sesión iniciada. Existen animales registrados en el sistema.

Postcondiciones: Reporte generado y mostrado en pantalla.

Datos (entrada / pantalla / salida):

Entrada:

- D1: rodeo: id_rodeo | todos (obligatorio)
- D2: tipo_reporte: string (obligatorio; opciones: actual, diario, semanal, quincenal, mensual, anual, personalizado)
- D3: fecha_referencia: date (obligatoria si D2 es un tipo fijo distinto de actual; punto desde el cual el sistema cuenta los días hacia atrás según D2. Si D2 = actual y no se informa, se usa la fecha actual)
- D4: desde / hasta: date, date (ambas obligatorias solo si D2 = personalizado; rango elegido libremente por el administrador)

Salida mostrada (reporte):

- S1: total_animales: int
- S2: Tabla con una fila por animal:
 - S2.1: caravana: string (caravana_cuig + caravana_nro_manejo)
 - S2.2: raza: string
 - S2.3: sexo: string
 - S2.4: edad: string (calculada desde fecha_nacimiento a la fecha actual)
 - S2.5: fecha_alta: date
 - S2.6: estado_actual: string
 - S2.7: rodeo_actual: string

Reglas de validación:

- R1: Si se indica un id_rodeo debe existir en el sistema; si se elige todos se incluyen todos los rodeos.
- R2: tipo_reporte debe ser una de las opciones indicadas.
- R3: Si D2 es un tipo fijo (actual, diario, semanal, quincenal, mensual, anual), el período se calcula como $[D3 - (N-1) \text{ días}, D3]$, donde N es fijo según D2 (actual=1, diario=1, semanal=7, quincenal=15, mensual=30, anual=365). Si D2 = actual y D3 no se informa, se toma la fecha actual. Si D2 = personalizado, el período es exactamente $[D4.\text{desde}, D4.\text{hasta}]$, y debe cumplirse fecha inicio menor o igual a fecha fin.
- R4: La edad se calcula desde fecha_nacimiento hasta la fecha actual.
- R5: Solo se incluyen animales activos en el reporte.

Flujo principal:

1. El sistema muestra el formulario con D1 y D2.
2. El administrador selecciona el rodeo o todos y el tipo de reporte. Si D2 es un tipo fijo distinto de actual, el sistema solicita D3. Si D2 = personalizado, el sistema solicita D4 (desde y hasta).
3. El administrador presiona Generar.
4. El sistema valida R1 y R2, calcula el período según R3 y recopila los animales del alcance seleccionado.
5. El sistema calcula la edad de cada animal y arma la tabla S2 con el total S1.
6. El sistema muestra el reporte en pantalla.

Excepciones / Alternativos:

- E1: Rodeo inexistente (R1): informar "No se encontró el rodeo seleccionado"; permitir corregir.
- E2: tipo_reporte inválido (R2): informar "El tipo de reporte ingresado no es válido"; permitir corregir.
- E3: D3 no informada siendo obligatoria (R3, tipo fijo distinto de actual): informar "Debe indicar la fecha del período para este tipo de reporte"; permitir corregir.
- E4: D4 incompleta siendo D2 = personalizado (R3): informar "Debe indicar desde y hasta para un reporte personalizado"; permitir corregir.

- E5: Rango inválido en personalizado (R3): informar "El período ingresado no es válido"; permitir corregir las fechas.
- E6: Sin resultados: informar "No se encontraron animales para el alcance y período seleccionados."

CU18 — Generar reporte de fechas importantes a recordar

Actor principal: Administrador

Propósito: Visualizar eventos sanitarios próximos y vencimientos de eventos sanitarios para un período determinado.

Precondiciones: Sesión iniciada. Existen eventos sanitarios registrados en el sistema.

Postcondiciones: Reporte generado y mostrado en pantalla.

Datos (entrada / pantalla / salida):

Entrada:

- D1: rodeo: id_rodeo | todos (opcional; filtra por los animales que pertenecen actualmente al rodeo)
- D2: tipo_reporte: string (obligatorio; opciones: diario, semanal, quincenal, mensual)
- D3: periodo: date o rango de fechas (obligatorio)

Salida mostrada (reporte):

- S1: Próximas aplicaciones sanitarias:
 - S1.1: destino: string (caravana: caravana_cuig + caravana_nro_manejo)
 - S1.2: tipo_evento: string
 - S1.3: medicamentos del detalle: string (si los hay)
 - S1.4: fecha_proxima_aplicacion: date
 - S1.5: responsable: string
- S2: Vencimientos de eventos sanitarios:
 - S2.1: destino: string (caravana: caravana_cuig + caravana_nro_manejo)
 - S2.2: tipo_evento: string
 - S2.3: medicamentos del detalle: string (si los hay)
 - S2.4: vigencia_hasta: date
 - S2.5: responsable: string

- S2.6: observaciones: string

Reglas de validación:

- R1: tipo_reporte debe ser una de las opciones indicadas; periodo con fechas válidas y coherentes; fecha inicio menor o igual a fecha fin.
- R2: Para S1 se incluyen eventos con fecha_proxima_aplicacion dentro del período.
- R3: Para S2 se incluyen eventos con vigencia_hasta dentro del período.

Flujo principal:

1. El sistema muestra el formulario con D1, D2 y D3.
2. El administrador selecciona el alcance, el tipo de reporte y el período; presiona Generar.
3. El sistema valida R1, R2 y R3.
4. El sistema arma S1 aplicando R2, y S2 aplicando R3.
5. El sistema muestra el reporte en pantalla.

Excepciones / Alternativos:

- E1: Sin eventos en el período: informar "No hay fechas programadas para el período seleccionado."
- E2: Período inválido (R1): informar "El período ingresado no es válido"; permitir corregir.
- E3: Rodeo inexistente (D1): informar "No se encontró el rodeo seleccionado"; permitir corregir.

CU19 — Generar reporte de evolución de peso

Actor principal: Administrador

Propósito: Analizar la variación de peso de los animales en un período determinado.

Precondiciones: Sesión iniciada. Existen registros de peso en el sistema.

Postcondiciones: Reporte generado y mostrado en pantalla.

Datos (entrada / pantalla / salida):

Entrada:

- D1: rodeo: id_rodeo | todos (obligatorio)
- D2: tipo_reporte: string (obligatorio; opciones: diario, semanal, quincenal, mensual)
- D3: periodo: date o rango de fechas (obligatorio)

- D4: caravana: caravana_cuig + caravana_nro_manejo (opcional; para filtrar por un animal específico)

Salida mostrada (reporte):

- S1: Tabla con una fila por animal:
 - S1.1: caravana: string (caravana_cuig + caravana_nro_manejo)
 - S1.2: fecha_inicial: date (primer pesaje del período)
 - S1.3: peso_inicial: decimal
 - S1.4: fecha_final: date (último pesaje del período)
 - S1.5: peso_final: decimal
 - S1.6: variacion_kg: decimal (peso_final menos peso_inicial; — si solo hay un registro)
 - S1.7: cantidad_registros: int
- S2: Detalle de pesajes al seleccionar un animal: fecha_pesaje y peso_kg por cada registro.
- S3: Gráfica de evolución de peso (disponible al filtrar por una sola caravana o al seleccionar ver gráfico).

Reglas de validación:

- R1: Si se indica un id_rodeo debe existir en el sistema; si se elige todos se incluyen todos los rodeos.
- R2: tipo_reporte debe ser una de las opciones indicadas; periodo con fechas válidas y coherentes; fecha inicio menor o igual a fecha fin.
- R3: Para calcular variacion_kg se requieren al menos dos registros de peso dentro del período.
- R4: Si solo hay un registro en el período se muestra la fila con variacion_kg = — y cantidad_registros = 1.
- R5: Si no hay registros en el período el animal no aparece en la tabla.

Flujo principal:

1. El sistema muestra el formulario con D1, D2, D3 y el filtro opcional D4.
2. El administrador completa los datos y presiona Generar.
3. El sistema valida R1 y R2.

4. El sistema reúne los registros de peso del período y arma S1 aplicando R3, R4 y R5.
5. El sistema muestra la tabla S1. Al seleccionar un animal muestra el detalle S2 y, si corresponde, la gráfica S3.

Excepciones / Alternativos:

- E1: Sin registros de peso en el período: informar "No se encontraron registros de peso para el período seleccionado."
- E2: Período inválido (R2): informar "El período ingresado no es válido"; permitir corregir.
- E3: Rodeo inexistente (R1): informar "No se encontró el rodeo seleccionado"; permitir corregir.

CU20 — Consultar usuarios

Actor principal: Administrador

Propósito: Visualizar la lista de usuarios del sistema con su rol asignado.

Precondiciones: Sesión iniciada. Actor con permisos de administración.

Postcondiciones: Lista de usuarios mostrada en pantalla.

Datos (entrada / pantalla / salida):

Entrada:

- D1: buscar: string (opcional; filtra por nombre de usuario o correo)
- D2: rol: string (opcional; filtra por rol)

Salida mostrada:

- S1: Tabla con una fila por usuario:
 - S1.1: nombre_usuario: string
 - S1.2: correo: string
 - S1.3: rol: string
- S2: total_usuarios: int

Reglas de validación:

- R1: Solo el Administrador puede acceder a esta pantalla.
- R2: Si se informa buscar se filtra por coincidencia parcial en nombre_usuario o correo.
- R3: Si se informa rol se muestran únicamente los usuarios con ese rol.

Flujo principal:

1. El sistema muestra la pantalla de administración de usuarios con los filtros D1 y D2.
2. El administrador completa los filtros opcionalmente y presiona Buscar.
3. El sistema valida R1, aplica R2 y R3 según corresponda y arma la tabla S1 con el total S2.
4. El sistema muestra la lista de usuarios.

Excepciones / Alternativos:

- E1: Sin resultados: informar "No se encontraron usuarios con los filtros seleccionados."
- E2: Error al cargar registros: informar "No se pudieron cargar los usuarios; intente nuevamente."

CU21 — Crear usuario

Actor principal: Administrador

Propósito: Dar de alta un nuevo usuario en el sistema.

Precondiciones: Sesión iniciada. Actor con permisos de administración.

Postcondiciones: Usuario creado, guardado y visible en la lista de usuarios.

Datos (entrada / pantalla / salida):

Entrada:

- D1: nombre_completo: string (obligatorio)
- D2: correo_electronico: string (obligatorio)
- D3: nombre_usuario: string (obligatorio)
- D4: contrasena: string (obligatorio)
- D5: rol: string (obligatorio; opciones: Administrador, Encargado de campo, Asesor técnico)

Salida tras guardar:

- S1: Mensaje: "Usuario creado correctamente."
- S2: Fila agregada en la lista con nombre_usuario, correo y rol.

Reglas de validación:

- R1: nombre_completo no vacío.

- R2: correo_electronico con formato válido y no repetido en el sistema.
- R3: nombre_usuario no vacío y no repetido en el sistema.
- R4: contraseña con longitud mínima de 8 caracteres.
- R5: rol debe ser una de las opciones permitidas.
- R6: Si el rol seleccionado es Administrador el sistema solicita doble confirmación antes de crear el usuario.

Flujo principal:

1. El sistema muestra el formulario con D1–D5.
2. El administrador completa los campos y presiona Guardar.
3. Si el rol seleccionado es Administrador, el sistema solicita doble confirmación según R6.
4. El administrador confirma.
5. El sistema valida R1–R5.
6. El sistema crea el usuario y muestra S1 y S2.

Excepciones / Alternativos:

- E1: Campos inválidos o incompletos (R1–R5): marcar el campo a corregir con mensaje específico; permitir reintentar.
- E2: Correo o nombre de usuario ya existente (R2, R3): informar "El correo o nombre de usuario ya está en uso"; permitir corregir.
- E3: Doble confirmación rechazada (R6): no se crea el usuario; volver al formulario.
- E4: Error al crear: informar "No se pudo crear el usuario; intente nuevamente."

CU22 — Editar usuario

Actor principal: Administrador

Propósito: Actualizar los datos o el rol de un usuario existente en el sistema.

Precondiciones: Sesión iniciada. Existe al menos un usuario registrado.

Postcondiciones: Datos del usuario actualizados y visibles en la lista de usuarios.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_usuario: int (resuelto por la pantalla)

- D2: nombre_completo: string (obligatorio)
- D3: correo_electronico: string (obligatorio)
- D4: nombre_usuario: string (obligatorio)
- D5: rol: string (obligatorio; opciones: Administrador, Encargado de campo, Asesor técnico)

Salida tras guardar:

- S1: Mensaje: "Usuario editado correctamente."
- S2: Fila actualizada en la lista con nombre_usuario, correo y rol.

Reglas de validación:

- R1: id_usuario debe existir en el sistema.
- R2: nombre_completo no vacío.
- R3: correo_electronico con formato válido y no repetido excluyendo al propio usuario.
- R4: nombre_usuario no vacío y no repetido excluyendo al propio usuario.
- R5: rol debe ser una de las opciones permitidas.
- R6: Si el cambio de rol es a Administrador el sistema solicita doble confirmación antes de guardar.
- R7: Un administrador no puede cambiar su propio rol.

Flujo principal:

1. El sistema muestra la lista de usuarios y permite seleccionar uno.
2. El administrador selecciona un usuario y presiona Editar.
3. El sistema muestra el formulario precompletado con D2–D5.
4. El administrador modifica los datos y presiona Guardar.
5. Si el nuevo rol es Administrador, el sistema solicita doble confirmación según R6.
6. El administrador confirma.
7. El sistema valida R1–R7.
8. El sistema actualiza los datos y muestra S1 y S2.

Excepciones / Alternativos:

- E1: Campos inválidos o duplicados (R2–R5): marcar el campo a corregir con mensaje específico; permitir reintentar.

- E2: Doble confirmación rechazada (R6): no se guardan cambios; volver al formulario.
- E3: Intento de editar su propio rol (R7): informar "No puede modificar su propio rol"; no se realizan cambios.
- E4: Error al editar: informar "No se pudo editar el usuario; intente nuevamente."

CU23 — Eliminar usuario

Actor principal: Administrador

Propósito: Dar de baja un usuario del sistema.

Precondiciones: Sesión iniciada. Existe al menos un usuario registrado distinto al administrador que realiza la operación.

Postcondiciones: Usuario eliminado y lista de usuarios actualizada.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_usuario: int (resuelto por la pantalla)

Confirmación:

- D2: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "Usuario eliminado correctamente."

Reglas de validación:

- R1: id_usuario debe existir en el sistema.
- R2: Se requiere confirmación explícita antes de eliminar (D2 = true).
- R3: Solo el Administrador puede realizar esta operación.
- R4: Un administrador no puede eliminarse a sí mismo.
- R5: No puede eliminarse al último administrador activo del sistema.

Flujo principal:

1. El sistema muestra la lista de usuarios.
2. El administrador selecciona un usuario y presiona Eliminar.
3. El sistema valida R3, R4 y R5 antes de continuar.
4. El sistema solicita confirmación (D2).

5. El administrador confirma.
6. El sistema valida R1 y R2, elimina al usuario y muestra S1.

Excepciones / Alternativos:

- E1: Operación cancelada (D2 = false): no se realizan cambios; volver a la lista de usuarios.
- E2: Intento de eliminarse a sí mismo (R4): informar "No puede eliminar su propio usuario"; no se realizan cambios.
- E3: Intento de eliminar al último administrador (R5): informar "No se puede eliminar al único administrador activo del sistema"; no se realizan cambios.
- E4: Error al eliminar: informar "No se pudo eliminar el usuario; intente nuevamente."

CU24 — Crear ingrediente

Actor principal: Asesor técnico / Administrador

Propósito: Agregar un ingrediente al catálogo para que pueda usarse en los planes alimenticios.

Precondiciones: Sesión iniciada. Actor con permisos de gestión de catálogo.

Postcondiciones: Ingrediente creado como activo y disponible para seleccionarse en planes alimenticios (CU9, CU11).

Datos (entrada / pantalla / salida):

Entrada:

- D1: nombre_ingrediente: string (obligatorio)
- D2: descripcion: string (opcional)
- D3: minerales: string (opcional)
- D4: energia_metabolizable: decimal (opcional)
- D5: proteina_bruta: decimal (opcional)
- D6: fibra_detergente_neutro: decimal (opcional)
- D7: unidad_medida: enum {kg, l, gr, ml, fardo, rollo} (obligatorio)
- D8: aditivos: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Ingrediente creado con éxito."
- S2: id_ingrediente: int asignado

- S3: Ingrediente visible en el catálogo con los datos cargados.

Reglas de validación:

- R1: nombre_ingrediente no vacío y no repetido entre los ingredientes activos (sin distinguir mayúsculas y minúsculas).
- R2: Si se informan valores nutricionales (D4–D6), deben ser mayores o iguales a 0.
- R3: unidad_medida debe ser una de las opciones permitidas.
- R4: El ingrediente se crea en estado activo.

Flujo principal:

1. El sistema muestra el formulario con D1–D8.
2. El actor completa los datos y presiona Guardar.
3. El sistema valida R1–R3.
4. El sistema crea el ingrediente aplicando R4.
5. El sistema muestra S1–S3.

Excepciones / Alternativos:

- E1: Nombre duplicado (R1): informar "Ya existe un ingrediente con ese nombre"; permitir corregir.
- E2: Valores nutricionales inválidos (R2): marcar el campo con mensaje específico; permitir corregir.
- E3: Unidad de medida inválida (R3): informar "Unidad de medida inválida" junto con las opciones permitidas; permitir corregir.

CU25 — Consultar ingredientes

Actor principal: Asesor técnico / Administrador

Propósito: Visualizar el catálogo de ingredientes y buscar uno por nombre.

Precondiciones: Sesión iniciada.

Postcondiciones: Lista de ingredientes mostrada en pantalla.

Datos (entrada / pantalla / salida):

Entrada:

- D1: buscar: string (opcional; filtra por nombre del ingrediente)

Salida mostrada:

- S1: Tabla con una fila por ingrediente:
 - S1.1: nombre_ingrediente: string
 - S1.2: descripcion: string
 - S1.3: minerales: string
 - S1.4: energia_metabolizable: decimal
 - S1.5: proteina_bruta: decimal
 - S1.6: fibra_detergente_neutro: decimal
 - S1.7: unidad_medida: string
 - S1.8: aditivos: string

Reglas de validación:

- R1: Solo se muestran ingredientes activos.
- R2: Si se informa buscar, se filtra por coincidencia parcial en nombre_ingrediente, sin distinguir mayúsculas y minúsculas.

Flujo principal:

1. El sistema muestra la pantalla del catálogo con el filtro D1.
2. El actor completa el filtro opcionalmente y presiona Buscar.
3. El sistema aplica R1 y R2 y arma la tabla S1.
4. El sistema muestra la lista.

Excepciones / Alternativos:

- E1: Catálogo vacío: informar "No hay ingredientes cargados."
- E2: Sin resultados para la búsqueda: informar "No existe un ingrediente con ese nombre."

CU26 — Editar ingrediente

Actor principal: Asesor técnico / Administrador

Propósito: Actualizar los datos de un ingrediente del catálogo.

Precondiciones: Sesión iniciada. Existe al menos un ingrediente activo.

Postcondiciones: Ingrediente actualizado. Los planes alimenticios que lo usan reflejan los datos nuevos, porque lo referencian y no guardan una copia.

Datos (entrada / pantalla / salida):**Entrada:**

- D1: id_ingrediente: int (resuelto por la pantalla)
- D2: nombre_ingrediente: string (obligatorio)
- D3: descripcion: string (opcional)
- D4: minerales: string (opcional)
- D5: energia_metabolizable: decimal (opcional)
- D6: proteina_bruta: decimal (opcional)
- D7: fibra_detergente_neutro: decimal (opcional)
- D8: unidad_medida: enum {kg, l, gr, ml, fardo, rollo} (obligatorio)
- D9: aditivos: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Ingrediente actualizado con éxito."
- S2: Fila actualizada en el catálogo.

Reglas de validación:

- R1: id_ingrediente debe existir y estar activo.
- R2: nombre_ingrediente no vacío y no repetido entre los ingredientes activos, excluyendo al propio ingrediente (sin distinguir mayúsculas y minúsculas).
- R3: Si se informan valores nutricionales (D5–D7), deben ser mayores o iguales a 0.
- R4: unidad_medida debe ser una de las opciones permitidas.

Flujo principal:

1. El sistema muestra el catálogo (CU25) y el actor selecciona un ingrediente.
2. El sistema muestra el formulario precompletado con D2–D9.
3. El actor modifica los datos y presiona Guardar.
4. El sistema valida R1–R4.
5. El sistema persiste los cambios y muestra S1 y S2.

Excepciones / Alternativos:

- E1: Ingrediente inexistente o inactivo (R1): informar "No existe un ingrediente con ese id"; volver al catálogo.

- E2: Nombre duplicado (R2): informar "Ya existe un ingrediente con ese nombre"; permitir corregir.
- E3: Valores nutricionales inválidos (R3): marcar el campo con mensaje específico; permitir corregir.
- E4: Unidad de medida inválida (R4): informar "Unidad de medida inválida" junto con las opciones permitidas; permitir corregir.

CU27 — Desactivar ingrediente

Actor principal: Asesor técnico / Administrador

Propósito: Dar de baja un ingrediente del catálogo sin perder su vínculo con los planes que ya lo usan.

Precondiciones: Sesión iniciada. Existe al menos un ingrediente activo.

Postcondiciones: Ingrediente marcado como inactivo. Deja de aparecer en el catálogo y no puede seleccionarse en planes nuevos ni agregarse en ediciones. Los planes existentes que lo contienen lo conservan.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_ingrediente: int (resuelto por la pantalla)

Confirmación:

- D2: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "Ingrediente desactivado con éxito."
- S2: Si el ingrediente está en uso: aviso con el nombre de cada plan alimenticio que lo contiene.

Reglas de validación:

- R1: id_ingrediente debe existir en el sistema.
- R2: El ingrediente debe estar activo.
- R3: La desactivación es lógica: el registro no se elimina y los detalles de planes que lo referencian se conservan.
- R4: Estar en uso en uno o más planes no impide la desactivación; solo genera el aviso S2.

- R5: Un plan que contenga un ingrediente inactivo no puede guardarse en CU11 hasta que ese componente se quite o reemplace.
- R6: Una vez desactivado, su nombre queda disponible para crear un nuevo ingrediente (CU24).

Flujo principal:

1. El sistema muestra el ingrediente seleccionado con el botón Desactivar.
2. El actor presiona Desactivar; el sistema solicita confirmación (D2).
3. El actor confirma.
4. El sistema valida R1 y R2, obtiene los planes que usan el ingrediente y aplica R3.
5. El sistema muestra S1 y, si corresponde, S2.

Excepciones / Alternativos:

- E1: Operación cancelada (D2 = false): no se realizan cambios; volver al catálogo.
- E2: Ingrediente inexistente (R1): informar "No existe un ingrediente con ese id"; volver al catálogo.
- E3: Ingrediente ya inactivo (R2): informar "El ingrediente ya se encuentra desactivado"; no se realizan cambios.

CU28 — Crear medicamento

Actor principal: Asesor técnico / Administrador

Propósito: Agregar un medicamento al catálogo para que pueda usarse en los eventos sanitarios.

Precondiciones: Sesión iniciada. Actor con permisos de gestión de catálogo.

Postcondiciones: Medicamento creado como activo y disponible para seleccionarse en eventos sanitarios (CU12).

Datos (entrada / pantalla / salida):

Entrada:

- D1: nombre: string (obligatorio)
- D2: descripcion: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Medicamento creado con éxito."
- S2: id_medicamento: int asignado

- S3: nombre: string
- S4: descripcion: string
- S5: estado: Activo

Reglas de validación:

- R1: nombre no vacío y no repetido en el sistema, incluidos los medicamentos inactivos (sin distinguir mayúsculas y minúsculas).
- R2: El medicamento se crea en estado activo.

Flujo principal:

1. El sistema muestra el formulario con D1 y D2.
2. El actor completa los datos y presiona Guardar.
3. El sistema valida R1.
4. El sistema crea el medicamento aplicando R2.
5. El sistema muestra S1–S5.

Excepciones / Alternativos:

- E1: Nombre duplicado (R1): informar "Ya existe un medicamento con ese nombre"; si el existente está inactivo, indicar que puede reactivarse (CU32); permitir corregir.

CU29 — Consultar medicamentos

Actor principal: Encargado de campo / Asesor técnico / Administrador

Propósito: Visualizar el catálogo de medicamentos y buscar uno por nombre.

Precondiciones: Sesión iniciada.

Postcondiciones: Lista de medicamentos mostrada en pantalla.

Datos (entrada / pantalla / salida):

Entrada:

- D1: buscar: string (opcional; filtra por nombre)
- D2: incluir_inactivos: boolean (opcional; por defecto false; solo Administrador)

Salida mostrada:

- S1: Tabla con una fila por medicamento:
 - S1.1: nombre: string
 - S1.2: descripcion: string

- S1.3: estado: string (Activo / Inactivo)

Reglas de validación:

- R1: Por defecto solo se muestran medicamentos activos.
- R2: Si se informa buscar, se filtra por coincidencia parcial en nombre, sin distinguir mayúsculas y minúsculas.
- R3: Solo el Administrador puede incluir medicamentos inactivos en el listado.

Flujo principal:

1. El sistema muestra la pantalla del catálogo con los filtros D1 y D2.
2. El actor completa los filtros opcionalmente y presiona Buscar.
3. El sistema aplica R1–R3 y arma la tabla S1.
4. El sistema muestra la lista.

Excepciones / Alternativos:

- E1: Catálogo vacío: informar "No hay medicamentos almacenados."
- E2: Sin resultados para la búsqueda: informar "No existe un medicamento con ese nombre."

CU30 — Editar medicamento

Actor principal: Asesor técnico / Administrador

Propósito: Actualizar los datos de un medicamento del catálogo.

Precondiciones: Sesión iniciada. Existe al menos un medicamento activo.

Postcondiciones: Medicamento actualizado. Los eventos sanitarios que lo referencian muestran los datos nuevos.

Datos (entrada / pantalla / salida):

Entrada:

- D1: id_medimento: int (resuelto por la pantalla)
- D2: nombre: string (obligatorio)
- D3: descripcion: string (opcional)

Salida tras guardar:

- S1: Mensaje: "Medicamento actualizado con éxito."
- S2: Fila actualizada en el catálogo.

Reglas de validación:

- R1: id_medicamento debe existir y estar activo.
- R2: nombre no vacío y no repetido en el sistema, incluidos los inactivos, excluyendo al propio medicamento (sin distinguir mayúsculas y minúsculas).

Flujo principal:

1. El sistema muestra el catálogo (CU29) y el actor selecciona un medicamento.
2. El sistema muestra el formulario precompletado con D2 y D3.
3. El actor modifica los datos y presiona Guardar.
4. El sistema valida R1 y R2.
5. El sistema persiste los cambios y muestra S1 y S2.

Excepciones / Alternativos:

- E1: Medicamento inexistente o inactivo (R1): informar "No existe un medicamento con ese id"; volver al catálogo.
- E2: Nombre duplicado (R2): informar "Ya existe otro medicamento con ese nombre"; permitir corregir.

CU31 — Desactivar medicamento

Actor principal: Asesor técnico / Administrador

Propósito: Dar de baja un medicamento del catálogo sin perder el historial sanitario que lo referencia.

Precondiciones: Sesión iniciada. Existe al menos un medicamento activo.

Postcondiciones: Medicamento marcado como inactivo. Deja de aparecer en el catálogo y no puede seleccionarse en nuevos eventos sanitarios. Los eventos ya registrados lo conservan.

Datos (entrada / pantalla / salida):**Entrada:**

- D1: id_medicamento: int (resuelto por la pantalla)

Confirmación:

- D2: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "Medicamento desactivado con éxito."

Reglas de validación:

- R1: id_medicamento debe existir en el sistema.
- R2: El medicamento debe estar activo.
- R3: La desactivación es lógica: el registro no se elimina y los detalles de medicamento de eventos sanitarios ya registrados se conservan.
- R4: Un medicamento inactivo no puede seleccionarse al registrar eventos sanitarios (CU12).

Flujo principal:

1. El sistema muestra el medicamento seleccionado con el botón Desactivar.
2. El actor presiona Desactivar; el sistema solicita confirmación (D2).
3. El actor confirma.
4. El sistema valida R1 y R2 y aplica R3.
5. El sistema muestra S1.

Excepciones / Alternativos:

- E1: Operación cancelada (D2 = false): no se realizan cambios; volver al catálogo.
- E2: Medicamento inexistente (R1): informar "No existe un medicamento con ese id"; volver al catálogo.
- E3: Medicamento ya inactivo (R2): informar "El medicamento ya se encuentra desactivado"; no se realizan cambios.

CU32 — Reactivar medicamento

Actor principal: Administrador

Propósito: Reactivar un medicamento dado de baja para que vuelva a estar disponible en el catálogo.

Precondiciones: Sesión iniciada. Actor con permisos de administración. Existe al menos un medicamento inactivo.

Postcondiciones: Medicamento activo, visible en el catálogo y seleccionable en eventos sanitarios.

Datos (entrada / pantalla / salida):**Entrada:**

- D1: id_medicamento: int (resuelto por la pantalla, desde CU29 con incluir_inactivos = true)

Confirmación:

- D2: confirmar: boolean (obligatorio)

Salida tras confirmar:

- S1: Mensaje: "El medicamento fue reactivado correctamente."
- S2: Fila del medicamento con estado: Activo.

Reglas de validación:

- R1: id_medicamento debe existir en el sistema.
- R2: El medicamento debe estar inactivo.
- R3: La reactivación restaura el estado a Activo sin modificar sus datos ni su historial.
- R4: Solo el Administrador puede realizar esta operación.

Flujo principal:

1. El sistema muestra el catálogo incluyendo inactivos (CU29).
2. El administrador selecciona un medicamento inactivo y presiona Reactivar.
3. El sistema solicita confirmación (D2).
4. El administrador confirma.
5. El sistema valida R1, R2 y R4 y aplica R3.
6. El sistema muestra S1 y S2.

Excepciones / Alternativos:

- E1: Medicamento inexistente (R1): informar "No existe un medicamento con ese id"; volver al catálogo.
- E2: Medicamento ya activo (R2): informar "El medicamento ya se encuentra activo"; no se realizan cambios.
- E3: Operación cancelada (D2 = false): no se realizan cambios; volver al catálogo.